

BÁO CÁO BÀI TẬP LỚN
Hệ thống huấn luyện Alphazero phân tán
với Bigdata pipeline

Tên HP:	Lưu trữ và xử lý dữ liệu lớn	
Mã HP:	IT4931	
Mã lớp:	168482	
Nhóm số:	15	
	Đinh Ngọc Tuyên	20235242
	Nguyễn Dương Việt Hùng	20235097
Thành viên:	Mai Quỳnh Hoa	20235082
	Phạm Thu Hà	20235070
	Đinh Bảo Long	20235141
GVHD:	TS. Trần Việt Trung	

Tóm tắt nội dung đề tài

Báo cáo này trình bày thiết kế kiến trúc và các bài học kinh nghiệm từ việc xây dựng nền tảng Data Lakehouse và MLOps phân tán cho dự án AI AlphaZero trên hạ tầng Bare-metal Kubernetes. Nhằm giải quyết các nút thắt cổ chai về hiệu năng và sự chia cắt môi trường công nghệ, dự án đã triển khai kiến trúc điện toán hợp nhất sử dụng Ray và Polars thay thế cho Apache Spark. Song song đó, toàn bộ dữ liệu huấn luyện được quản lý theo tiêu chuẩn Medallion trên MinIO và bảo vệ bởi giao thức Delta Lake, giúp đảm bảo tính toàn vẹn ACID và khả năng chống trùng lặp.

Các đúc kết kỹ thuật quan trọng chỉ ra rằng: việc thiết kế mã nguồn "bất khả tri hạ tầng" (tự nhận diện tài nguyên CPU/GPU), phân tách hoàn toàn lớp tính toán khỏi lớp lưu trữ, và áp dụng cơ chế phục hồi lỗi (Checkpointing) hướng chất lượng là chìa khóa sống còn. Nhờ vậy, nền tảng không chỉ tự động hóa khép kín vòng đời học máy mà còn đạt được khả năng tự đàn hồi, sẵn sàng mở rộng quy mô từ máy tính cá nhân lên các cụm máy chủ công nghiệp mà không cần thay đổi cấu trúc mã nguồn.

MỤC LỤC

CHƯƠNG 1. ĐỊNH NGHĨA BÀI TOÁN.....	6
1.1 Bài toán lựa chọn: Hệ thống huấn luyện Alphazero phân tán với Bigdata pipeline.....	6
1.2 Phân tích mức độ phù hợp với dữ liệu lớn.....	7
1.3 Phạm vi và giới hạn dự án.....	7
CHƯƠNG 2. KIẾN TRÚC VÀ THIẾT KẾ.....	9
2.1 Kiến trúc tổng thể.....	9
2.2 Các thành phần và vai trò.....	10
2.2.1 Lớp Xử lý và Lưu trữ Dữ liệu (Processing & Storage Layer) ...	10
2.2.2 Lớp Điện toán Phân tán (Distributed Compute Layer).....	11
2.2.3 Lớp Trình diễn và Quản lý vòng đời (Serving & Presentation / MLOps Layer).....	12
2.2.4 Lớp Hạ tầng và Điều phối (Infrastructure/Monitoring Layer)...	12
2.3 Sơ đồ luồng dữ liệu	13
2.3.1 Giai đoạn Ingestion (Thu thập dữ liệu thô):.....	13
2.3.2 Giai đoạn Medallion Processing (Xử lý và Tinh sạch dữ liệu):. 13	
2.3.3 Giai đoạn Self-play (Tự chơi sinh dữ liệu):	13
2.3.4 Giai đoạn Distributed Training & MLOps (Huấn luyện phân tán & Ghi nhận):	14
CHƯƠNG 3. CHI TIẾT TRIỂN KHAI.....	15
3.1 Cấu trúc mã nguồn	15
3.1.1 Thư mục <code>docker/</code>	15
3.1.2 Thư mục <code>k8s/</code>	16
3.1.3 Thư mục <code>src/</code>	19
3.2 Cài đặt hệ thống	20
3.2.1 Yêu cầu phần cứng.....	21
3.2.2 Yêu cầu phần mềm nền tảng	21
3.2.3 Quy hoạch cổng mạng nội bộ.....	21
3.2.4 Quy trình cài đặt.....	21
CHƯƠNG 4. TỔNG KẾT	23
4.1 Tổng kết hệ thống.....	23
4.2 Hạn chế.....	23
4.3 Bài học thu được	24

4.3.1	Bài học 1: Xử lý văn bản PGN lớn và tính toán vẹn.....	24
4.3.2	Xử lý dữ liệu với Spark.....	25
4.3.3	Quản lí vòng đời dữ liệu sinh ra từ self-play	26
4.3.4	Lưu trữ dữ liệu	26
4.3.5	Tích hợp hệ thống.....	27
4.3.6	Tối ưu hóa hiệu năng.....	27
4.3.7	Giám sát, gỡ lỗi	28
4.3.8	Mở rộng (Scaling)	29
4.3.9	Chất lượng – kiểm thử.....	29
4.3.10	Bảo mật, quản trị	30
4.3.11	Chịu lỗi.....	31

DANH MỤC HÌNH VẼ

Hình 2.1 Minh họa kiến trúc DeltaLake (Nguồn: Internet)	10
Hình 2.2 Sơ đồ luồng dữ liệu	13
Hình 3.1 Cấu trúc thư mục mã nguồn	15
Hình 3.2 Cấu trúc thư mục	15
Hình 3.3 Cấu trúc thư mục	16
Hình 3.4 Cấu trúc thư mục	19

CHƯƠNG 1. ĐỊNH NGHĨA BÀI TOÁN

1.1 Bài toán lựa chọn: Hệ thống huấn luyện Alphazero phân tán với Bigdata pipeline

Việc lựa chọn xây dựng một nền tảng huấn luyện phân tán cho mô hình AlphaZero chơi cờ vua không chỉ là một bài toán về thuật toán Trí tuệ nhân tạo (AI), mà cốt lõi là một bài toán phức tạp về Kỹ thuật Hệ thống (Systems Engineering) và Vận hành Học máy (MLOps).

Kể từ năm 2017, kiến trúc AlphaZero do DeepMind giới thiệu đã định hình lại cách chúng ta tiếp cận các bài toán có không gian trạng thái khổng lồ. Bằng cách kết hợp Mạng Nơ-ron (Neural Networks) với Thuật toán Tìm kiếm cây Monte Carlo (MCTS) thông qua cơ chế tự chơi (Self-play Reinforcement Learning), AI có thể tự học từ con số không. Tuy nhiên, để đẩy nhanh quá trình hội tụ của mô hình ở giai đoạn đầu (Supervised Learning/Khởi động), việc tận dụng kho dữ liệu ván đấu lịch sử khổng lồ của con người là vô cùng cần thiết. Đằng sau sự kết hợp này là một "khoảng cách tử thần" trong khâu triển khai thực tế, đòi hỏi một hệ thống có khả năng tiêu hóa dữ liệu thô quy mô lớn và điều phối tính toán song song.

Hiện thực và Thách thức (Reality):

- Nút thắt trong khâu Kỹ thuật Dữ liệu (Data Engineering Bottleneck): Các nhà nghiên cứu AI thường mất tới 80% thời gian cho việc làm sạch và chuẩn bị dữ liệu. Việc xử lý hàng triệu ván cờ từ tập PGN của Chess.com không thể giải quyết bằng một đoạn mã Python `read_file()` đơn giản. Nếu làm thủ công hoặc dùng kiến trúc xử lý tuần tự, hệ thống sẽ gặp tình trạng thắt cổ chai, tràn RAM (Out-of-Memory) và tốn hàng tuần chỉ để chuẩn bị dữ liệu trước khi AI kịp học bất cứ thứ gì.
- Khoảng cách tài nguyên và Giới hạn mở rộng (Resource Gap): Các tổ chức lớn sử dụng hàng ngàn máy chủ GPU/TPU. Trong khi đó, với các hệ thống tự dựng, một máy trạm đơn lẻ sẽ kiệt sức khi phải vừa chạy ETL, vừa duy trì hàng ngàn luồng mô phỏng MCTS, vừa cập nhật trọng số Neural Network. Việc không có cơ chế điều phối và phân tán tính toán sẽ khiến dự án đi vào ngõ cụt.
- Nợ kỹ thuật trong quản lý vòng đời (MLOps Technical Debt): Huấn luyện mô hình là một quá trình kéo dài và dễ đứt gãy. Việc thiếu vắng một hệ sinh thái giám sát chuẩn mực sẽ dẫn đến hiện tượng: mất dấu các tệp trọng số (weights), không thể truy vết lại các siêu tham số (hyperparameters) đã dùng, và không thể phục hồi lại trạng thái huấn luyện khi xảy ra sự cố phần cứng đột xuất.

Dự án này hướng tới việc kiến trúc và triển khai một nền tảng Dữ liệu lớn xử lý theo lô và MLOps phân tán (Distributed Batch Data & MLOps Pipeline). Nền tảng này tập trung vào khả năng tự động hóa luồng ETL dữ liệu ván đấu hàng tháng từ Chess.com, kết hợp với luồng dữ liệu sinh ra từ Self-play, lưu trữ quy

mô lớn có tính chịu lỗi cao, và điều phối cụm máy chủ huấn luyện AlphaZero một cách linh hoạt trên hạ tầng Bare-metal Kubernetes.

1.2 Phân tích mức độ phù hợp với dữ liệu lớn

Để giải quyết các thách thức vận hành nêu trên, hệ thống hiện tại mang đầy đủ các đặc tính cốt lõi của bài toán Dữ liệu lớn, qua đó chứng minh sự cần thiết phải sử dụng các công nghệ Big Data:

- *Volume (Khối lượng) & High-Throughput Batch (Xử lý lô thông lượng cao)*: Khối lượng dữ liệu lưu trữ hàng tháng từ Chess.com lên tới hàng chục Gigabyte văn bản nén, chứa hàng trăm triệu nước đi. Quá trình sinh dữ liệu Self-play cũng tạo ra ma trận trạng thái khổng lồ. Hệ thống yêu cầu năng lực tiêu hóa, chuyển đổi và ghi nhận một lượng dữ liệu đồ sộ vào Data Lakehouse (Apache Iceberg) trong một khung thời gian giới hạn của mỗi chu kỳ cập nhật.
- *Variety (Đa dạng định dạng)*: Nền tảng phải "tiêu hóa" đồng thời 3 luồng định dạng hoàn toàn khác biệt:
 - + Dữ liệu văn bản đặc thù của cờ vua (PGN - Portable Game Notation).
 - + Dữ liệu dạng bảng có cấu trúc được tối ưu hóa cho phân tích máy học.
 - + Tập nhị phân đồ sộ chứa trọng số mô hình AI (.pt, .weights).
- *Veracity (Độ tin cậy và Nhất quán)*: Độ tin cậy của dữ liệu là yếu tố sống còn trong hệ thống học máy phân tán. Một tệp Checkpoint bị phân mảnh do lỗi mạng, hoặc một lô dữ liệu bị ghi đè ngẫu nhiên (Race Condition) khi nhiều Node cùng truy cập sẽ làm sập toàn bộ quy trình. Hệ thống bắt buộc phải áp dụng kho lưu trữ dạng Object (MinIO) để đảm bảo dữ liệu được nhân bản chính xác 100% ở cấp độ vật lý.

1.3 Phạm vi và giới hạn dự án

Phạm vi thực hiện :

- Xây dựng Batch Data Pipeline theo kiến trúc Medallion: Tự động hóa hoàn toàn việc thu thập dữ liệu chơi cờ từ chess.com, xử lý dữ liệu qua 3 tầng (Bronze - Silver - Gold), từ khâu tiếp nhận dữ liệu PGN thô nguyên bản cho đến khi chuyển đổi thành ma trận số học sẵn sàng cho huấn luyện mô hình.
- Kiến trúc Lưu trữ & Quản lý: Sử dụng MinIO (S3-compatible) làm lõi Object Storage kết hợp với chuẩn Delta Lake để đảm bảo tính ACID (Atomicity, Consistency, Isolation, Durability) khi ghi dữ liệu theo lô.
- Động cơ Tính toán & MLOps: Triển khai cụm tính toán phân tán Ray Cluster để thực thi luồng ETL và huấn luyện AI. Quản lý vòng đời mô hình (ML Lifecycle) bằng MLflow Tracking và backend PostgreSQL.
- Hạ tầng Vật lý (Infrastructure): Triển khai toàn bộ dưới dạng mã (IaC) trên cụm Bare-metal Kubernetes (K3s) quy mô 2 Nodes (1 Master/Storage, 1 Worker).

Giới hạn:

- Hệ thống không sử dụng kiến trúc xử lý luồng thời gian thực (như Apache Kafka) mà tập trung hoàn toàn vào tối ưu hóa chi phí và tài nguyên phần cứng cho kiến trúc Batch Processing.
- Không đặt mục tiêu huấn luyện một siêu AI đạt hệ số Elo vượt qua các Đại kiện tướng (điều này đòi hỏi cụm siêu máy tính GPU khổng lồ chạy liên tục trong nhiều tháng).

CHƯƠNG 2. KIẾN TRÚC VÀ THIẾT KẾ

2.1 Kiến trúc tổng thể

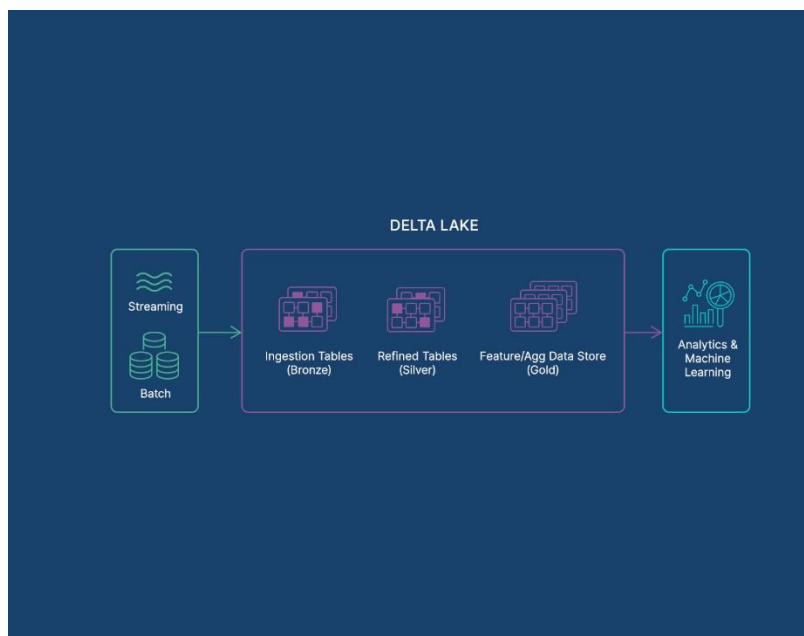
Hệ thống huấn luyện AI cờ vua phân tán (AlphaZero) được xây dựng dựa trên mô hình Kiến trúc Data Lakehouse kết hợp Điện toán phân tán nhằm đáp ứng đồng thời hai yêu cầu quan trọng: khả năng tiêu hóa lượng dữ liệu lịch sử khổng lồ (tính thông lượng cao) và năng lực tính toán song song các mô phỏng AI phức tạp. Khác với kiến trúc thời gian thực (như Lambda), kiến trúc xử lý theo lô (Batch Processing) này đặc biệt phù hợp với các hệ thống Big Data & AI, nơi dữ liệu ván đấu và log tự chơi (self-play) phát sinh với dung lượng lớn theo chu kỳ, đòi hỏi tối ưu hóa tài nguyên phần cứng cho khâu huấn luyện Mạng Nơ-ron.

Toàn bộ hệ thống được chia thành các lớp chức năng chính gồm: Ingestion Layer, Processing & Storage Layer, Distributed Compute Layer, MLOps Layer và lớp hỗ trợ Infrastructure. Các thành phần này phối hợp với nhau để tạo thành một pipeline khép kín từ khâu thu thập dữ liệu thô, xử lý tinh sạch, huấn luyện AI cho đến quản lý vòng đời mô hình học máy.

Chi tiết về cấu trúc phân lớp của hệ thống:

- **Ingestion Layer:** Chịu trách nhiệm trích xuất dữ liệu ván đấu lịch sử khổng lồ (định dạng PGN) từ Chess.com và dữ liệu sinh ra từ quá trình AI tự chơi. Dữ liệu được thu thập theo từng lô (batch) và đẩy trực tiếp vào vùng chứa nguyên bản của nền tảng lưu trữ.
- **Processing & Storage Layer (Data Lakehouse):**
 - + *Lớp Lưu trữ (Storage):* Sử dụng MinIO làm trung tâm dữ liệu Object Storage, tổ chức chặt chẽ theo mô hình Medallion 3 tầng (Bronze - Silver - Gold). Tầng Silver đặc biệt tích hợp giao thức Delta Lake để bảo vệ tính toàn vẹn giao dịch (ACID) khi ghi dữ liệu song song. Tầng Gold lưu trữ dữ liệu dưới định dạng Tensor (.npz) tối ưu cho Mạng Nơ-ron.
 - + *Lớp Xử lý (Processing):* Ứng dụng thư viện tốc độ cao Polars để phân tích cú pháp (parsing) văn bản, trích xuất đặc trưng và chuẩn hóa dữ liệu trực tiếp trên bộ nhớ (in-memory) trước khi ghi xuống hồ dữ liệu.
- **Distributed Compute Layer:** Đóng vai trò là "động cơ" của toàn hệ thống thông qua cụm Ray Cluster. Ray điều phối tính toán phân tán cho tất cả nhiệm vụ: thu thập dữ liệu, huấn luyện mô hình, self-play.
- **MLOps & Presentation Layer:** Quản lý trung tâm thông qua MLflow kết hợp với cơ sở dữ liệu backend PostgreSQL. Lớp này chịu trách nhiệm lưu vết các siêu tham số (hyperparameters), giám sát độ đo (Loss/Accuracy) và tự động hóa việc đăng ký, đánh dấu các phiên bản mô hình tốt nhất (Checkpoints). Giao diện Web UI của MLflow đóng vai trò là tầng Presentation để kỹ sư theo dõi quá trình huấn luyện.
- **Infrastructure/Monitoring:** Toàn bộ hệ thống được triển khai vi dịch vụ trên môi trường Bare-metal K3s Kubernetes thông qua Docker Container. Do không sử dụng bộ lưu trữ khối phân tán, hệ thống áp dụng chiến lược cấp phát lưu trữ cục bộ (Local Provisioning): ghim các thành phần có trạng thái

(MinIO, Postgres) vào ổ đĩa vật lý cố định, trong khi các tác vụ tính toán (Ray Workers) được mở rộng tự do, linh hoạt nhằm đảm bảo khả năng phục hồi khi hệ thống xảy ra hiện tượng tràn bộ nhớ (OOM).



Hình 2.1 Minh họa kiến trúc DeltaLake (Nguồn: Internet)

2.2 Các thành phần và vai trò

Hệ thống được thiết kế theo tư duy kiến trúc hướng dịch vụ (Microservices), phân bổ các công cụ chuyên biệt vào từng lớp chức năng. Mỗi thành phần không chỉ thực hiện một nhiệm vụ độc lập mà còn mang trong mình cơ chế sinh tồn đặc thù để đối phó với rủi ro trong môi trường điện toán phân tán.

2.2.1 Lớp Xử lý và Lưu trữ Dữ liệu (Processing & Storage Layer)

Lớp này đóng vai trò là "Data Lakehouse", chịu trách nhiệm tiêu hóa, biến đổi và lưu trữ an toàn khối lượng dữ liệu vụn cò khổng lồ.

2.2.1.1. MinIO (S3-Compatible Object Storage)

- Là kho tàng trữ vật lý trung tâm (Data Lake), nơi tiếp nhận PGN thô, chứa các bảng Delta Lake (Silver), tệp Tensor .npz (Gold) và kho trọng số mô hình (Checkpoints).
- Cơ chế xử lý bài toán Big Data:
 - + Bảo vệ dữ liệu bằng Mã hóa xóa (Erasure Coding): Do hệ thống không sử dụng lớp ảo hóa ổ đĩa phân tán ở cấp độ K8s, MinIO tự bảo vệ sự an toàn của dữ liệu bằng thuật toán Erasure Coding. Thay vì nhân bản toàn bộ file (mirroring) gây tốn dung lượng, MinIO chia nhỏ các tệp PGN và Checkpoint thành các khối dữ liệu (Data blocks) và khối dự phòng (Parity blocks). Nếu một phần ổ đĩa vật lý bị hỏng (bad sector), MinIO vẫn có thể tái tạo lại dữ liệu gốc một cách hoàn hảo thông qua các toán tử ma trận, đảm bảo tính sẵn sàng cao (High Availability).

- + Thông lượng cao (High Throughput): Kiến trúc hướng đối tượng cho phép các cụm tính toán đọc/ghi song song trực tiếp bỏ qua nút thắt I/O của hệ điều hành truyền thống.

2.2.1.2. Delta Lake

- Là giao thức quản lý bảng dữ liệu (Table Format) được đặt trên nền MinIO tại tầng Silver.
- Cơ chế xử lý bài toán Big Data - Kiểm soát đồng thời: Khi hàng chục tác vụ của Ray cùng ghi kết quả xử lý vào MinIO, rủi ro ghi đè (Race Condition) và hỏng tệp là cực kỳ cao. Delta Lake giải quyết vấn đề này bằng Nhật ký giao dịch (Transaction Log). Cơ chế này khóa lạc quan (Optimistic Concurrency Control), đảm bảo tính nguyên tử (Atomicity): một lô dữ liệu hoặc được ghi thành công toàn bộ, hoặc bị từ chối nếu có xung đột, triệt tiêu hoàn toàn rủi ro sinh ra "dữ liệu rác".

2.2.1.3. Polars

- Là động cơ xử lý DataFrame siêu tốc được nhúng bên trong lớp Điện toán.
- Cơ chế xử lý: Được viết bằng Rust, Polars bỏ qua giới hạn đơn luồng của Python (GIL). Nó biên dịch các tác vụ chuyển đổi dữ liệu thành một Biểu đồ thực thi vật lý (Physical Query Plan) và chạy đa luồng trực tiếp trên bộ nhớ RAM, giúp việc phân tích cú pháp (parsing) hàng triệu dòng văn bản PGN nhanh hơn gấp nhiều lần so với Pandas.

2.2.2 Lớp Điện toán Phân tán (Distributed Compute Layer)

Đóng vai trò là "Động cơ" của hệ thống, gánh vác các luồng công việc nặng nhất mà một máy chủ đơn lẻ không thể thực hiện.

2.2.2.1. Cụm Ray (Ray Head & Ray Workers)

- Là nền tảng điều phối tính toán phân tán. Điều phối cả luồng xử lý dữ liệu (ETL) kết hợp với Polars và luồng huấn luyện mạng nơ-ron.-
- Cơ chế xử lý bài toán Big Data:
 - + Cơ chế điều phối công việc phi tập trung (Distributed Scheduling): Ray không đẩy toàn bộ gánh nặng lên Node Master. Hệ thống duy trì một máy chủ trạng thái trung tâm (Global Control Store - GCS) để theo dõi toàn cụm, nhưng mỗi Worker đều chạy một trình quản lý cục bộ gọi là Raylet. Khi luồng ETL sinh ra hàng ngàn tác vụ phân tích PGN nhỏ (@ray.remote), Head Node sẽ dựa vào báo cáo tài nguyên (CPU/RAM trống) từ các Raylet để đẩy task về đúng máy đang rảnh, đảm bảo tối đa hóa hiệu suất phần cứng (Load Balancing).
 - + Chia sẻ bộ nhớ tốc độ cao (Distributed Object Store): Khi xử lý dữ liệu lớn, việc copy dữ liệu giữa các máy tính qua mạng LAN sẽ làm sập hệ thống. Ray sử dụng cơ chế Plasma Store (bộ nhớ dùng chung). Nếu hai tiến trình trên cùng một Worker cần đọc cùng một tệp .npz, chúng đọc chung một vùng nhớ vật lý (Zero-copy read) thay vì nhân bản dữ liệu ra RAM, giúp tiết kiệm triệt để tài nguyên.

- + Phục hồi dựa trên Checkpoint (Checkpoint-based Fault Tolerance): Khi hiện tượng tràn bộ nhớ (OOM) làm sập một Node trong lúc huấn luyện, đứt gãy vòng giao tiếp cụm là không thể tránh khỏi. Thay vì bùng bít lỗi, Ray dựa vào cơ chế ghi Checkpoints liên tục xuống MinIO. Khi khởi động lại, Worker nạp ngược bản Checkpoint gần nhất để tiếp tục, triệt tiêu rủi ro mất trắng dữ liệu huấn luyện.

2.2.3 Lớp Trình diễn và Quản lý vòng đời (Serving & Presentation / MLOps Layer)

Cung cấp giao diện tương tác và bộ nhớ trạng thái cho việc huấn luyện dài hạn.

2.2.3.1. MLflow & PostgreSQL

- Đóng vai trò quản lý vòng đời MLOps.
- Cơ chế xử lý: Để tránh rủi ro "Não chia cắt" (Split-brain) khi các Worker nạp nhầm phiên bản mô hình, MLflow áp dụng thiết kế phân tách trạng thái. Tập vật lý dung lượng lớn lưu ở MinIO, nhưng toàn bộ siêu dữ liệu, lịch sử độ đo (Metrics) và đường dẫn định tuyến tệp được ghi vào cấu trúc quan hệ có tính ACID cao của PostgreSQL.

2.2.3.2. Giao diện Trực quan hóa (Dashboards)

- Cung cấp "tầm nhìn" cho kỹ sư để tháo gỡ trạng thái "hộp đen" của hệ thống phân tán. Tích hợp Ray Dashboard để theo dõi cấu trúc liên kết tác vụ (Task Topology), tình trạng CPU/RAM của từng Worker theo thời gian thực, và MLflow UI để vẽ biểu đồ giám sát sự hội tụ của Mạng Nơ-ron.

2.2.4 Lớp Hạ tầng và Điều phối (Infrastructure/Monitoring Layer)

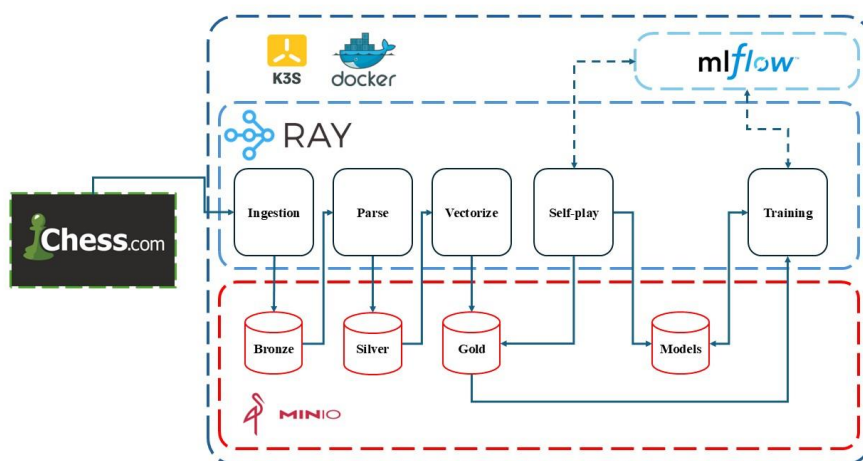
2.2.4.1. Kubernetes (Bare-metal K3s) & Local Storage Provisioning

- Là hệ điều hành ảo hóa, điều phối vi dịch vụ và quản lý mạng nội bộ (CoreDNS).
- Cơ chế xử lý: K3s áp dụng chiến lược quy hoạch không gian lưu trữ (Local Provisioning). Các thành phần Stateful (MinIO, Postgres) được ghim chặt vào đĩa vật lý của máy chủ gốc bằng NodeSelector. Đối với rủi ro đứt gãy tiến trình, K3s cung cấp cơ chế phục hồi môi trường linh hoạt (Self-healing). Khi một Worker bị "bắn hạ" do OOMKilled, K3s tự động dọn dẹp và cấp phát lại một Pod sạch, cung cấp lại một môi trường trống cho Ray Head tiếp tục phân việc.

2.2.4.2. Docker

- Là tiêu chuẩn đóng gói môi trường (Containerization).
- Cơ chế xử lý: Xóa bỏ rủi ro "xung đột thư viện phân tán" (Dependency Hell). Mọi tiến trình từ Head đến Worker đều khởi chạy từ một Docker Image bất biến, đảm bảo sự nhất quán bit-to-bit của môi trường (PyTorch, Ray, Polars) trên tất cả các máy vật lý, là tiền đề sinh tồn cho khả năng nhân rộng cụm.

2.3 Sơ đồ luồng dữ liệu



Hình 2.2 Sơ đồ luồng dữ liệu

Luồng dữ liệu đi qua hệ thống được vận hành khép kín và tự động điều phối toàn diện bởi cụm Ray Cluster, trải qua chu trình 4 giai đoạn rõ ràng:

2.3.1 Giai đoạn Ingestion (Thu thập dữ liệu thô):

- Tác vụ phân tán data-ingestion chạy trên cụm Ray chủ động thiết lập kết nối và gọi Chess.com Public API để lọc và thu thập các tệp biên bản ván đấu lịch sử.
- Tải xuống dữ liệu ván cờ dưới định dạng văn bản PGN nén (.pgn.gz) nhằm tối ưu hóa băng thông mạng và tốc độ truyền tải.
- Đẩy toàn bộ các tệp dữ liệu thô này vào vùng chứa (bucket) Bronze trên nền tảng Object Storage MinIO để tàng trữ nguyên bản, thiết lập nền móng đầu tiên cho kiến trúc Data Lakehouse.

2.3.2 Giai đoạn Medallion Processing (Xử lý và Tinh sạch dữ liệu):

- Tiến trình Ray data-parsing đọc các tệp PGN thô từ tầng Bronze, sử dụng sức mạnh của thư viện Polars thực hiện bóc tách song song các đặc trưng cốt lõi (chuỗi nước đi, hệ số Elo, kết quả ván đấu) trực tiếp trên bộ nhớ (In-memory) của các Worker Nodes.
- Kết quả sau khi làm sạch và loại bỏ ván đấu lỗi được ghi đồng thời xuống vùng chứa Silver dưới định dạng Delta Table, áp dụng cơ chế ghi gia tăng (incremental append) để bảo vệ tính giao dịch ACID và ngăn ngừa xung đột dữ liệu.
- Tiếp tục chạy tác vụ vectorization đọc dữ liệu bảng từ tầng Silver, thực hiện mã hóa các nước đi cờ vua thành các mảng ma trận số học đa chiều và nén chặt thành tệp Tensor dưới định dạng .npz để lưu trữ trực tiếp vào tầng Gold.

2.3.3 Giai đoạn Self-play (Tự chơi sinh dữ liệu):

- Định kỳ theo chu kỳ huấn luyện, cụm Ray khởi chạy luồng tác vụ self-play phân tán trên các Worker nhằm chủ động tạo ra nguồn dữ liệu chất lượng cao. Mỗi Worker Pod gửi tín hiệu API tới MLflow Tracking Server để truy vấn đường dẫn của phiên bản mô hình tốt nhất hiện tại (@champion). MLflow

- tra cứu dữ liệu trong PostgreSQL backend và trả về tọa độ chính xác để Worker tải tệp trọng số nhị phân tương ứng từ vùng chứa Models của MinIO.
- Các Worker thực thi thuật toán tìm kiếm cây Monte Carlo (MCTS) cho hai thực thể AI tự đối đầu với nhau, sinh ra các ván đấu mới, vector hóa trực tiếp trên RAM và đẩy thẳng kết quả vào vùng chứa Gold để tích hợp vào tập dữ liệu huấn luyện.

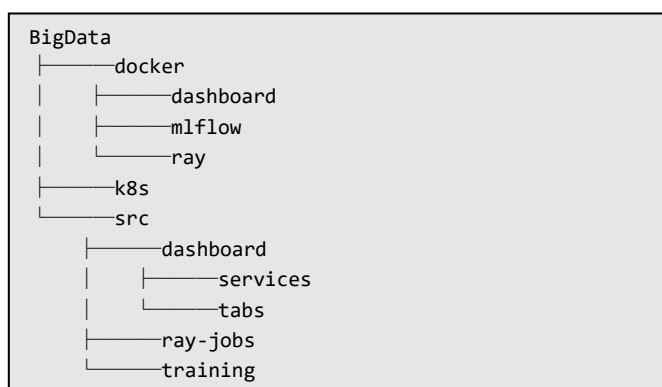
2.3.4 Giai đoạn Distributed Training & MLOps (Huấn luyện phân tán & Ghi nhận):

- Tiến trình Ray distributed-training nạp đồng loạt khối lượng lớn dữ liệu ma trận Tensor .npz từ tầng Gold (bao gồm cả dữ liệu lịch sử từ Chess.com và dữ liệu tự chơi) phối hợp với tệp trọng số mô hình nền tảng.
- Hệ thống sử dụng framework học sâu PyTorch triển khai huấn luyện phân tán trên toàn cụm dưới sự điều phối của Ray Head, tính toán song song và cập nhật liên tục các trọng số mới cho Mạng Nơ-ron qua các chu kỳ (Epochs).
- Xuất các tệp trọng số mô hình mới (Model Checkpoints) lưu trữ trực tiếp vào vùng chứa Models trên MinIO để bảo toàn thành quả tính toán kể cả khi xảy ra sự cố tràn bộ nhớ (OOM). Đồng thời, Ray Worker bắn siêu dữ liệu (Loss, Accuracy, Hyperparameters) cùng đường dẫn S3 của tệp trọng số vừa lưu về MLflow Tracking Server để cập nhật lên Dashboard trực quan hóa, thuận tiện cho việc theo dõi hệ thống.

CHƯƠNG 3. CHI TIẾT TRIỂN KHAI

3.1 Cấu trúc mã nguồn

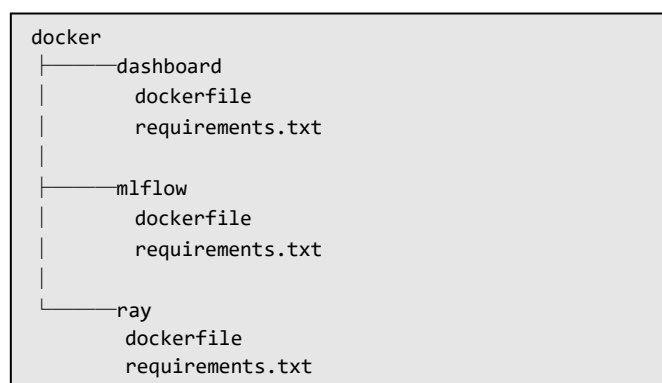
Cấu trúc tổng quan các thư mục trong project:



Hình 3.1 Cấu trúc thư mục mã nguồn

Chi tiết mã nguồn xem tại <https://github.com/nguyenduongviethung/BigData.git>

3.1.1 Thư mục `docker/`



Hình 3.2 Cấu trúc thư mục

Thư mục `docker/` đóng vai trò là trọng tâm quản lý cấu hình môi trường của toàn bộ hệ thống. Nhiệm vụ cốt lõi của thư mục này bao gồm:

- Đảm bảo mã nguồn Python (AI, ETL) khi thực thi trên máy cá nhân hay trên các cụm máy chủ vật lý phân tán (Bare-metal K3s) đều chạy trên một môi trường bit-to-bit giống nhau tuyệt đối.
- Thay vì tạo ra một Docker Image công kênh duy nhất chứa tất cả mọi thứ, thư mục này phân tách môi trường thành 3 Image độc lập. Chiến lược này giúp tối ưu hóa dung lượng lưu trữ trên Registry, giảm băng thông mạng khi K3s kéo (Pull) Image, và gia tăng tốc độ khởi động (Cold-start time) của các Pod.

Mỗi thư mục con đại diện cho một phân hệ độc lập trong hệ thống Lakehouse & MLOps, được định nghĩa qua cặp file cốt lõi:

3.1.1.1. Phân hệ Động cơ điện toán phân tán

Đây là môi trường nặng nhất hệ thống, gánh vác toàn bộ luồng xử lý Big Data và huấn luyện AI.

- **dockerfile**: Định nghĩa hệ điều hành nền (thường là Ubuntu tích hợp sẵn các driver tính toán hiệu năng cao như CUDA nếu có GPU). File này chịu trách nhiệm thiết lập các biến môi trường cho cụm Ray, cấu hình cổng giao tiếp mạng (Ports) phục vụ cho giao tiếp phân tán, và cài đặt các công cụ hệ thống cần thiết cho việc đọc/ghi dữ liệu lớn qua mạng LAN.
- **requirements.txt**: Khai báo các thư viện chuyên biệt phục vụ cho luồng dữ liệu và học máy như **ray**, **polars**, **torch**, **s3fs**, ...

3.1.1.2. Phân hệ Quản lý Vòng đời Máy học

Môi trường này được tối ưu hóa ở mức siêu nhẹ (Lightweight), chỉ phục vụ việc lưu vết và điều hướng mô hình.

- **dockerfile**: Sử dụng một Image nền Python tối giản . File này cấu hình lệnh khởi chạy cho MLflow Server, mở cổng mặc định 5000 và thiết lập kết nối tới kho lưu trữ trọng số vật lý (MinIO) cùng kho siêu dữ liệu (PostgreSQL).
- **requirements.txt**: Chỉ chứa các thư viện phục vụ ghi sổ cái và quản trị, không chứa các thư viện tính toán nặng: **mlflow**, **boto3**, **psycog2-binary**

3.1.1.3. Phân hệ Trực quan hóa Công việc

Môi trường cô lập dành riêng cho giao diện quản trị và theo dõi tiến độ của các kỹ sư vận hành.

- **dockerfile**: Đóng gói mã nguồn giao diện Dashboard, cấu hình cổng mạng để người dùng cuối truy cập từ trình duyệt, đảm bảo giao diện độc lập hoàn toàn với cụm điện toán để không bị ảnh hưởng tài nguyên khi hệ thống đang chạy tác vụ nặng.
- **requirements.txt**: Chứa các framework phục vụ xây dựng và render giao diện Web giao tiếp nhanh qua API và các thư viện vẽ biểu đồ tiến độ công việc, giúp trực quan hóa cấu trúc liên kết tác vụ đang thực thi ngầm trên cụm Ray.

3.1.2 Thư mục **k8s/**

```
k8s
  dashboard.yaml
  ingress.yaml
  minio.yaml
  mlflow.yaml
  postgres.yaml
  ray-cluster.yaml
```

Hình 3.3 Cấu trúc thư mục

Thư mục **k8s/** giữ vai trò là Bản thiết kế điều phối hạ tầng (Infrastructure Orchestration Blueprint). Toàn bộ tài nguyên phần cứng vật lý (CPU, RAM, Ổ

cứng) được ảo hóa và phân phối tập trung thông qua các tệp YAML này nhằm đạt được các mục đích cốt lõi:

- Hiện thực hóa kiến trúc phân tách Tính toán và Lưu trữ: Định nghĩa rõ ràng ranh giới giữa các dịch vụ lưu giữ trạng thái vật lý (Stateful) và các dịch vụ tính toán thuần túy (Stateless).
- Xây dựng cơ chế tự phục hồi (Self-healing) và Giới hạn tài nguyên: Thiết lập các ngưỡng tài nguyên (Requests/Limits) để giám lỏng tiến trình, bảo vệ cụm vật lý khỏi thảm họa tràn bộ nhớ (OOM) và tự động tái tạo môi trường sạch ngay khi xảy ra sự cố.
- Quản lý mạng nội bộ và điều hướng giao tiếp: Cấu hình mạng lưới Service và Ingress để các thành phần có thể tìm thấy và giao tiếp với nhau qua DNS nội bộ của K3s, đồng thời mở cổng ra mạng bên ngoài cho người giám sát.

3.1.2.1. Cấu hình kho Object Storage

- Vai trò: Khởi tạo lớp hạ tầng lưu trữ Data Lakehouse (Bronze, Silver, Gold) và kho chứa các phiên bản Model Checkpoints.
- Cơ chế đặc thù: Tích hợp bộ ba thành phần K8s để quản lý lưu trữ: PersistentVolumeClaim yêu cầu cấp phát 10GB dung lượng cục bộ qua K3s; Deployment điều phối 1 Pod MinIO duy nhất gắn ổ đĩa vào thư mục `/data` kèm cơ chế nạp tài khoản bảo mật từ `alphazero-secrets`; và Service kiểu LoadBalancer giúp mở cổng 9000 (API nội bộ) cùng cổng 9001 (Console quản trị) ra mạng bên ngoài.

3.1.2.2. Cấu hình Cơ sở dữ liệu Metadata

- Vai trò: Triển khai hệ quản trị cơ sở dữ liệu PostgreSQL làm Backend Store cho MLflow Tracking Server.
- Cơ chế đặc thù: Tương tự MinIO, đây là một thành phần Stateful quan trọng lưu trữ toàn bộ "sổ cái" siêu tham số và độ đo AI. File này định nghĩa một Service kiểu ClusterIP mở cổng nội bộ 5432, kèm theo cấu hình ghim Node vật lý để đảm bảo dữ liệu lịch sử huấn luyện không bị mất trắng khi Pod bị khởi động lại.

3.1.2.3. Cấu hình MLOps Server

- Vai trò: Triển khai dịch vụ MLflow Tracking Server phục vụ việc quản lý vòng đời mô hình.
- Cơ chế đặc thù: Tệp này khai báo một Deployment phi trạng thái (Stateless). Các thông tin kết nối nhạy cảm như tài khoản MinIO và chuỗi kết nối Postgres được trích xuất an toàn từ Kubernetes Secrets và bơm vào Container dưới dạng biến môi trường. Khi khởi chạy, Pod này phụ thuộc hoàn toàn vào trạng thái sẵn sàng của hai file `minio.yaml` và `postgres.yaml`.

3.1.2.4. Cấu hình cụm Điện toán phân tán

- Vai trò: Thiết lập hệ thống tính toán phân tán Ray phục vụ huấn luyện
- Cơ chế đặc thù: Hệ thống sử dụng cơ chế kết nối qua Service nội bộ để các Worker tự động đăng ký với Head mà không cần cấu hình IP tĩnh.

Đặc biệt, cấu hình `emptyDir` với `medium: Memory` tạo ổ đĩa chia sẻ `/dev/shm` trên RAM, cho phép các tiến trình trao đổi dữ liệu huấn luyện với tốc độ cực cao, tối ưu hóa hiệu năng cho mô hình đòi hỏi xử lý dữ liệu lớn. Việc sử dụng `secretRef` giúp bảo mật các biến môi trường, đảm bảo tính nhất quán và an toàn cho toàn bộ cụm node trong quá trình vận hành.

3.1.2.5. Cấu hình Giao diện trực quan hóa công việc

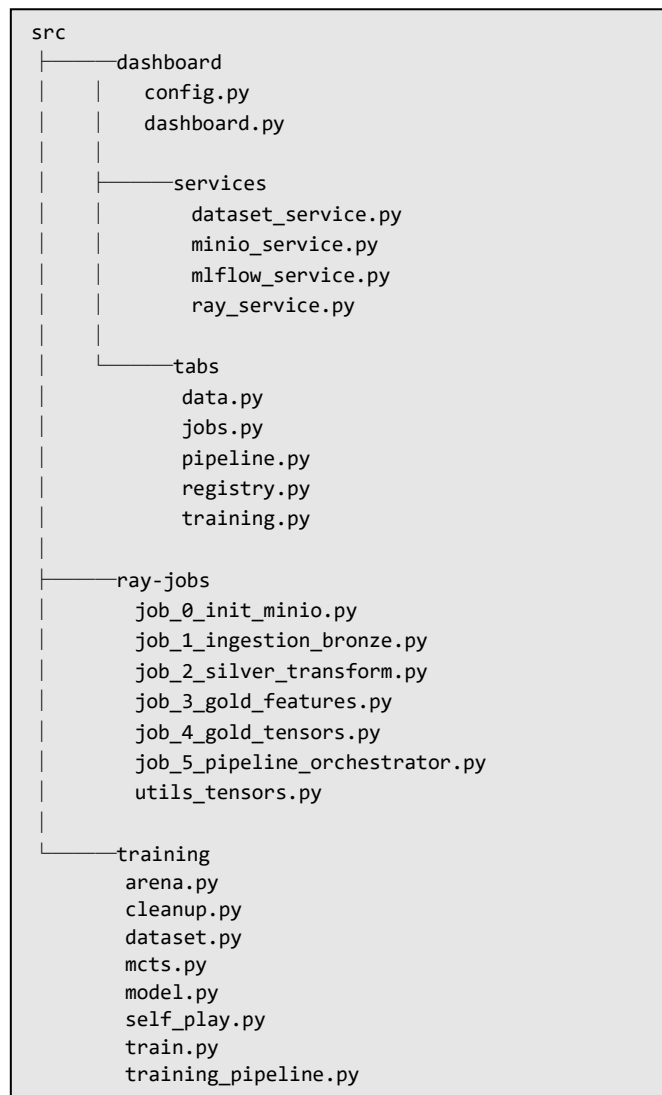
- Vai trò: Triển khai ứng dụng Dashboard chuyên dụng để theo dõi tiến độ các luồng công việc.
- Cơ chế đặc thù: Khai báo một dịch vụ Stateless gọn nhẹ, sử dụng chính xác Docker Image đã build. Dịch vụ này kết nối trực tiếp với API của Ray Head và MLflow để tổng hợp dữ liệu tiến độ và hiển thị lên giao diện Web UI.

3.1.2.6. Cấu hình Cổng điều hướng phân tuyến

- Vai trò: Lớp cửa ngõ duy nhất (Reverse Proxy/Load Balancer) đứng trước cụm K3s để định tuyến lưu lượng mạng từ bên ngoài vào các dịch vụ nội bộ.
- Cơ chế đặc thù: Sử dụng bộ điều phối Ingress mặc định của K3s (Traefik) để phân tách các yêu cầu HTTP dựa trên tên miền phụ (Subdomains) hoặc đường dẫn (Paths).

3.1.3 Thư mục `src/`

Thư mục `src/` chứa toàn bộ logic nghiệp vụ, thuật toán trí tuệ nhân tạo và các



Hình 3.4 Cấu trúc thư mục

đường ống xử lý dữ liệu của hệ thống. Đây là nơi chứa mã nguồn Python để cụm Ray điều phối và thực thi.

3.1.3.1. Phân hệ Giao diện giám sát

- Vai trò: Cung cấp trung tâm điều khiển trực quan (Control Center) cho kỹ sư hệ thống nhằm tháo gỡ trạng thái "hộp đen" của cụm phân tán.
- Chi tiết các file chức năng:
 - + `config.py` & `dashboard.py`: Khởi tạo ứng dụng nền tảng UI (Streamlit/Dash) và thiết lập cấu hình kết nối mạng.
 - + Thư mục `services/`: Lớp trung gian chịu trách nhiệm gọi API và truy vấn trạng thái hệ thống
 - + Thư mục `tabs/`: Định nghĩa các tab giao diện riêng biệt: `data.py` (quản lý tệp), `jobs.py` (giám sát tác vụ), `pipeline.py` (vận hành luồng), `registry.py` (quản lý phiên bản mô hình) và `training.py` (hiển thị biểu đồ học máy).

3.1.3.2. Phân hệ Đường ống dữ liệu Lakehouse

- Vai trò: Định nghĩa các tác vụ xử lý lô tuần tự để biến đổi dữ liệu ván cờ qua kiến trúc Medallion.
- Chi tiết các file chức năng:
 - + `job_0_init_minio.py`: Khởi chạy đầu tiên để thiết lập cấu trúc các Bucket nguyên bản trên MinIO.
 - + `job_1_ingestion_bronze.py`: Tác vụ phân tán gọi API để cào dữ liệu ván đấu thô về tầng Bronze.
 - + `job_2_silver_transform.py`: Kích hoạt Polars để parse văn bản PGN song song và ghi bảng Delta Table có kiểm soát giao dịch ACID xuống tầng Silver.
 - + `job_3_gold_features.py` & `job_4_gold_tensors.py`: Trích xuất đặc trưng cờ vua nâng cao và mã hóa chúng thành các ma trận toán học Tensor, nén lại dưới dạng tệp .npz tại tầng Gold.
 - + `job_5_pipeline_orchestrator.py`: Bộ tổng chỉ huy (Orchestrator) chịu trách nhiệm lập lịch và kích hoạt tuần tự chuỗi Job từ 0 đến 4 một cách tự động.
 - + `utils_tensors.py`: Chứa các hàm toán học hỗ trợ cho việc biến đổi cấu trúc mảng đa chiều.

3.1.3.3. Phân hệ huấn luyện lõi AlphaZero

- Vai trò: Triển khai chu trình học tăng cường (Reinforcement Learning) khép kín của thuật toán AlphaZero.
- Chi tiết các file chức năng:
 - + `model.py`: Định nghĩa kiến trúc Mạng nơ-ron kép (Dual-headed Neural Network) gồm Policy head (gợi ý nước đi) và Value head (đánh giá cục diện) bằng PyTorch.
 - + `mcts.py`: Thuật toán Tìm kiếm cây Monte Carlo tối ưu hóa bằng luồng phân tán để tìm ra nước đi tốt nhất dựa trên gợi ý của Mạng nơ-ron.
 - + `self_play.py`: Tiến trình điều phối các Worker cho mô hình AI tự đối đầu với chính nó để liên tục sinh ra tập dữ liệu huấn luyện mới chất lượng cao chất lượng cao đẩy về tầng Gold.
 - + `train.py` & `training_pipeline.py`: Lõi huấn luyện phân tán, nạp dữ liệu Tensor từ tầng Gold để cập nhật trọng số (weights) mới cho mạng nơ-ron qua từng chu kỳ.
 - + `arena.py`: Đấu trường thẩm định, cho mô hình mới sinh ra thách đấu với mô hình đương kim vô địch (@champion). Chỉ khi mô hình mới thắng với tỷ lệ quy định thì mới được công nhận và đăng ký phiên bản mới lên MLflow.
 - + `dataset.py` & `cleanup.py`: Quản lý luồng nạp dữ liệu tối ưu cho GPU/CPU và dọn dẹp các mô hình cũ.

3.2 Cài đặt hệ thống

3.2.1 Yêu cầu phần cứng

Hệ thống yêu cầu tối thiểu một cụm phân tán gồm 01 Node Master và ít nhất 01 Node Worker. Do đặc thù thuật toán MCTS và PyTorch Training tiêu thụ tài nguyên cực lớn, cấu hình RAM được khuyến nghị cao để giảm thiểu thảm họa tràn bộ nhớ (OOM):

Thành phần phần cứng	Cấu hình tối thiểu	Cấu hình khuyến nghị	Vai trò trong hệ thống
Node Master	• CPU: 4 Cores • RAM: 8 GB • Disk: 40 GB SSD	• CPU: 8 Cores • RAM: 16 GB • Disk: 100 GB NVMe	Chạy K3s Control Plane, Ray Head Node, MinIO (Data Lake), PostgreSQL và MLflow.
Node Worker (Mỗi Node)	• CPU: 8 Cores • RAM: 16 GB • Disk: 30 GB SSD	• CPU: 16 Cores • RAM: 32 GB+ hoặc có GPU • Disk: 50 GB NVMe	Chạy Ray Worker thực thi luồng song song ETL Polars, tính toán Self-play và Training PyTorch.
Băng thông mạng	100 Mbps (Nội bộ)	1 Gbps (Mạng LAN nội bộ)	Đảm bảo tốc độ đồng bộ mã nguồn (runtime_env) và trao đổi dữ liệu mảng Tensor lớn qua Plasma Store.

3.2.2 Yêu cầu phần mềm nền tảng

Toàn bộ các máy chủ vật lý trong cụm phải được cài đặt đồng bộ các lớp phần mềm nền sau:

- Hệ điều hành: Ubuntu 22.04 LTS hoặc Ubuntu 24.04 LTS (Kiến trúc x86_64).
- Container Engine: Docker Engine phiên bản v24.0 hoặc mới hơn (Dùng để build Image).
- Bộ điều phối cụm: K3s v1.28+ (Phần phân phối Kubernetes siêu nhẹ cho Bare-metal).

3.2.3 Quy hoạch cổng mạng nội bộ

Hệ thống yêu cầu mở (Allow) các cổng mạng sau trên tường lửa của cụm K3s để các Microservices thông tin với nhau:

- 9000 / 9001: Cổng API dữ liệu và Console của MinIO.
- 5432: Cổng kết nối cơ sở dữ liệu PostgreSQL Backend.
- 5000: Cổng gọi API của MLflow Tracking Server.
- 8265: Cổng giao diện và API điều phối của Ray Head Node.
- 10001 - 10010: Dải cổng giao tiếp nội bộ giữa Ray Head và Ray Workers.

3.2.4 Quy trình cài đặt

- Bước 1: Khởi tạo cluster

```
k3d cluster create alphazero-cluster \
  --servers 1 \
  --agents 1 \
  -p "80:80@loadbalancer" \
  -p "10001:10001@loadbalancer" \
  --registry-create alphazero-registry:5050
```

- Bước 2: Đóng gói và đẩy Image:

```
docker buildx build -f docker/ray/Dockerfile -t
localhost:5050/alphazero-node:v1 --push .
docker buildx build -f docker/mlflow/Dockerfile -t
localhost:5050/custom-mlflow:v1 --push .
docker buildx build -f docker/dashboard/Dockerfile -t
localhost:5050/chess-dashboard:v24 --push .
```

- Bước 3: Tạo `.env` theo mẫu `.env.example` và tạo kubernetes secret từ các biến môi trường được định nghĩa trong `.env`

```
kubectl apply -f k8s/minio.yaml
kubectl apply -f k8s/ray-cluster.yaml
kubectl apply -f k8s/postgres.yaml
kubectl apply -f k8s/mlflow.yaml
kubectl apply -f k8s/dashboard.yaml
kubectl apply -f k8s/ingress.yaml
```

- Để thực thi file mã nguồn:

```
ray job submit --address http://ray.local:8265 \
  --working-dir ./src \
  python <file_path>
```

CHƯƠNG 4. TỔNG KẾT

4.1 Tổng kết hệ thống

Dự án đã nghiên cứu và hiện thực hóa thành công một nền tảng Data Lakehouse kết hợp MLOps phân tán, phục vụ trọn vẹn vòng đời học máy khép kín của mô hình trí tuệ nhân tạo AlphaZero. Vượt qua những rào cản về giới hạn phần cứng và độ phức tạp của hệ thống phân tán, đề tài đã đạt được các thành tựu cốt lõi sau:

- Xây dựng kiến trúc điện toán hợp nhất (Unified Compute Engine): Thay thế thành công các công cụ ETL truyền thống (Apache Spark) bằng nền tảng Ray kết hợp thư viện Polars. Giải pháp này giúp đồng nhất toàn bộ đường ống xử lý dữ liệu (Data Pipeline) và huấn luyện mô hình (ML Pipeline) trên cùng một ngôn ngữ Python, loại bỏ hoàn toàn độ trễ do ranh giới môi trường và tận dụng tối đa năng lực xử lý đa luồng trên bộ nhớ (In-memory Processing).
- Triển khai Data Lakehouse an toàn và mở rộng: Xây dựng thành công kho lưu trữ 3 tầng (Bronze - Silver - Gold) theo kiến trúc Medallion trên nền tảng MinIO Object Storage. Việc áp dụng định dạng Delta Lake kết hợp cơ chế khử trùng lặp lũy đẳng (Idempotency) hai tầng đã bảo vệ hệ thống khỏi thảm họa mất tính toàn vẹn dữ liệu (Data Corruption) và xử lý triệt để các rủi ro ghi đè khi có sự cố mạng.
- Tự động hóa luồng MLOps và tự phục hồi: Ứng dụng thành công thư viện Ray Train và MLflow để quản lý vòng đời mô hình. Hệ thống sở hữu khả năng tự nhận diện tài nguyên phần cứng (CPU/GPU) để co giãn tải trọng, kết hợp cơ chế Cửa sổ trượt (Sliding Window) kiểm soát bộ nhớ và chiến lược lưu Checkpoint hướng chất lượng, giúp AI luôn tự đứng dậy từ trạng thái thông minh nhất sau mọi thảm họa tràn bộ nhớ (OOM).
- Chuẩn hóa quy trình triển khai bằng IaC (Infrastructure as Code): Đóng gói toàn bộ cấu hình môi trường, mạng nội bộ và các vi dịch vụ (Microservices) dưới dạng mã hóa trên cụm Bare-metal Kubernetes (K3s). Hệ thống đảm bảo tính di động cao, cho phép triển khai mượt mà từ môi trường phát triển cá nhân lên cụm máy chủ công nghiệp chỉ bằng các biến môi trường, mà không cần can thiệp vào mã nguồn lõi.

4.2 Hạn chế

Bên cạnh những kết quả đạt được, do giới hạn về mặt tài nguyên vật lý và thời gian nghiên cứu, hệ thống kiến trúc hiện tại vẫn tồn tại một số đánh đổi (trade-offs) và rủi ro kỹ thuật:

- Rủi ro về tính sẵn sàng cao của hệ thống lưu trữ (Storage Single Point of Failure): Việc sử dụng Local Path Provisioner để ghim cơ sở dữ liệu (PostgreSQL) và Data Lake (MinIO) vào ổ cứng của Master Node giúp tối ưu hóa tốc độ I/O nhưng lại hy sinh tính sẵn sàng cao. Nếu máy chủ vật lý Master gặp sự cố phần cứng nghiêm trọng, hệ thống không có cơ chế lưu trữ

- khối phân tán (như Ceph hay Longhorn) để tự động sao chép và di dời dữ liệu sang Node khác.
- Giới hạn trong khả năng cảnh báo chủ động (Proactive Alerting): Hệ thống hiện tại đã có khả năng quan sát (Observability) tốt thông qua các Dashboard của Ray và MLflow. Tuy nhiên, luồng giám sát vẫn mang tính thụ động (kỹ sư phải tự mở giao diện để xem). Thiếu vắng một cơ chế cảnh báo tự động (Alerting System như Prometheus/Alertmanager) để gửi thông báo tức thời (qua Email, Slack) khi luồng ETL thất bại hoặc khi mô hình AI có dấu hiệu suy thoái nghiêm trọng (Model Degradation).
 - Nút thắt cổ chai tại luồng mô phỏng MCTS: Quá trình tự chơi (Self-play) bằng thuật toán Monte Carlo Tree Search đòi hỏi năng lực tính toán thuần túy của CPU cực kỳ lớn. Dù kiến trúc đã hỗ trợ mở rộng ngang (Horizontal Scaling), tỷ lệ sinh dữ liệu ván đấu mới đôi khi vẫn chưa theo kịp tốc độ tiêu thụ dữ liệu của GPU trong luồng huấn luyện mạng nơ-ron, dẫn đến hiện tượng GPU phải "chờ" dữ liệu ở một số thời điểm.
 - Hiện tượng OOM (Out of memory): Việc chạy hệ thống gồm K3s, ray-cluster, minio, mlflow yêu cầu rất cao về RAM, với giới hạn phần cứng trên máy local (8-12GB RAM), vừa phải duy trì hệ thống, vừa phải đảm bảo hoạt động của hệ điều hành rất dễ gây ra hiện tượng OOM, khiến ray tự động kill toàn bộ tiến trình đang chạy.
 - Chính sách bảo mật nội bộ (Internal Security): Mặc dù đã tách biệt việc cấp quyền truy cập mô hình qua "Con trỏ JSON" và sử dụng K8s Secrets, lưu lượng giao tiếp nội bộ giữa các luồng Ray Worker và cơ sở dữ liệu bên trong cụm K3s hiện vẫn truyền tải dưới dạng bản rõ (Plain text). Trong môi trường doanh nghiệp quy mô lớn hơn, cần bổ sung các lớp mã hóa mạng nội bộ (mTLS) để tuân thủ tuyệt đối các tiêu chuẩn bảo mật Zero-Trust.

4.3 Bài học thu được

4.3.1 Bài học 1: Xử lý văn bản PGN lớn và tính toàn vẹn

4.3.1.1. Mô tả vấn đề

- Bối cảnh và nền tảng: Dữ liệu ván cờ lịch sử từ Chess.com được tải về dưới định dạng văn bản thô (PGN).
- Thách thức gặp phải: Các tệp PGN có kích thước lớn. Việc parse (phân tích cú pháp) văn bản thô tốn rất nhiều tài nguyên. Nếu các Ray Worker cùng ghi kết quả xử lý vào một bảng lưu trữ cùng lúc, hệ thống sẽ gặp lỗi ghi đè (Race Condition).
- Tác động đến hệ thống: Mất mát dữ liệu, sinh ra dữ liệu rác, và luồng ETL bị gián đoạn do hỏng file lưu trữ.

4.3.1.2. Cách tiếp cận đã thử

- Cách 1: Sử dụng định dạng CSV/Parquet thông thường lưu trên MinIO. Dễ triển khai nhưng không hỗ trợ ghi đồng thời an toàn (ACID).
- Cách 2: Sử dụng PostgreSQL để lưu lịch sử ván đấu. Đánh đổi: Dễ nghẽn cổ chai I/O khi Insert hàng triệu dòng cùng lúc.

4.3.1.3. Giải pháp cuối cùng

- Triển khai Delta Lake tại tầng Silver. Khi các Ray Worker trích xuất xong PGN, dữ liệu được ghi xuống Delta Table. Giao thức này sử dụng Nhật ký giao dịch (Transaction Log) với cơ chế Khóa lạc quan (Optimistic Concurrency Control). Nếu lô dữ liệu nào bị xung đột, nó sẽ bị từ chối và ghi lại, đảm bảo tính nguyên tử (Atomicity).

4.3.1.4. Điểm rút ra

- Hiểu biết kỹ thuật: Object Storage (MinIO) kết hợp với Table Format (Delta Lake) là tiêu chuẩn bắt buộc khi nhiều tiến trình phân tán cùng ghi dữ liệu (Multi-writer scenario).

4.3.2 Xử lý dữ liệu với Spark

4.3.2.1. Mô tả vấn đề

- Bối cảnh và nền tảng: Hệ thống cần xử lý ETL (bóc tách PGN, làm sạch, vector hóa) với khối lượng dữ liệu lớn trước khi đưa vào huấn luyện mô hình AlphaZero.
- Thách thức gặp phải: Tiêu chuẩn công nghiệp cho Big Data ETL thường là Apache Spark. Tuy nhiên, hệ sinh thái AI/ML (PyTorch, thuật toán MCTS) lại vận hành hoàn toàn bằng Python.
- Tác động đến hệ thống: Phải duy trì và cấp phát tài nguyên cho hai "động cơ" không lồ chạy song song trên cụm K3s, không thể duy trì với phần cứng hạn chế.

4.3.2.2. Cách tiếp cận đã thử

- Cách 1: Dùng Apache Spark (PySpark) cho giai đoạn Bronze - Silver, và dùng môi trường Python thuần cho giai đoạn Gold - Train. Phải vận hành hai giao diện giám sát khác nhau, và dữ liệu bắt buộc phải ghi xuống đĩa vật lý (MinIO) ở mỗi chặng chuyển giao giữa hai engine, gây nghẽn cổ chai I/O.

4.3.2.3. Giải pháp cuối cùng

- Quyết định loại bỏ hoàn toàn Apache Spark khỏi kiến trúc. Sử dụng Ray làm "Động cơ điện toán hợp nhất" (Unified Compute Engine) cho cả hai nhiệm vụ. Để thay thế khả năng xử lý dữ liệu bảng (DataFrame) của Spark, hệ thống nhúng thư viện Polars (viết bằng Rust) vào bên trong các tác vụ `@ray.remote`.

4.3.2.4. Điểm rút ra

- Hiểu biết kỹ thuật & Đề xuất: Spark vẫn là vua trong xử lý dữ liệu truyền thống. Nhưng với các dự án AI/Deep Learning hiện đại đặt trọng tâm vào Python, chiến lược dùng Ray + Polars để thay thế Spark giúp kiến trúc sư "một mũi tên trúng hai đích": vừa giải quyết bài toán Data Engineering (ETL), vừa tối ưu hóa bài toán ML Engineering (Huấn luyện phân tán) trên cùng một cụm phần cứng.

4.3.3 Quản lí vòng đời dữ liệu sinh ra từ self-play

4.3.3.1. Mô tả vấn đề

- Bối cảnh và nền tảng: Không giống hệ thống truyền thống chỉ đọc dữ liệu có sẵn, mô hình AlphaZero liên tục sinh ra dữ liệu mới thông qua chu trình tự chơi (Self-play MCTS).
- Thách thức gặp phải: Dữ liệu mới được sinh ra liên tục dưới dạng các ma trận nước đi. Làm sao để hòa trộn luồng dữ liệu này với dữ liệu lịch sử một cách đồng nhất.
- Tác động đến hệ thống: Nếu ghi rời rạc, luồng Training sẽ phải đọc hàng ngàn file nhỏ, gây quá tải I/O (Small Files Problem).

4.3.3.2. Cách tiếp cận đã thử

- Cách 1: Ghi mỗi ván cờ tự chơi thành 1 file .npz riêng. Đánh đổi: Hệ thống sụp đổ vì MinIO bị cạn kiệt Inodes và quá tải Request.
- Cách 2: Ghi vào bộ nhớ đệm rồi đẩy theo lô.

4.3.3.3. Giải pháp cuối cùng

- Thiết lập tiến trình `self_play.py` tích lũy ván đấu trên bộ nhớ đệm Plasma Store của Ray. Khi đạt đủ một kích thước lô (Batch Size - ví dụ 10,000 ván), Ray Worker mới nén lại thành một tệp .npz lớn đẩy xuống MinIO.

4.3.3.4. Điểm rút ra

- Ngay cả trong xử lý luồng liên tục, cơ chế Micro-batching (gom lô nhỏ) trước khi ghi xuống Object Storage là bắt buộc để tránh lỗi "Small Files Problem".

4.3.4 Lưu trữ dữ liệu

4.3.4.1. Mô tả vấn đề

- Bối cảnh và nền tảng: Hệ thống thực hiện dịch chuyển và cấu trúc hóa dữ liệu từ tầng thô (Bronze - các tệp tin văn bản PGN) sang tầng sạch (Silver - các bảng dữ liệu ván cờ) trên hệ thống Object Storage (MinIO) bằng sức mạnh tính toán phân tán của cụm Ray.
- Thách thức gặp phải: Các hệ thống Object Storage bản chất chỉ là các kho chứa "Key-Value" thụ động, không có cơ chế quản lý giao dịch (Transactions). Khi nhiều Ray Worker Node cùng tham gia bóc tách dữ liệu và đồng thời ghi kết quả xuống một vùng lưu trữ (Multi-writer scenario), hệ thống rất dễ rơi vào trạng thái xung đột dữ liệu (Race Conditions). Hơn nữa, nếu cấu trúc dữ liệu bị thay đổi đột xuất (Schema drift) giữa các lô ván cờ thu thập từ các nguồn khác nhau, các tác vụ đọc dữ liệu ở phân hệ sau sẽ bị sụp đổ.
- Tác động đến hệ thống: Dữ liệu tại tầng Silver dễ bị trùng lặp, mất tính toàn vẹn (Data Corruption), để lại các tệp tin rác ghi dở dang khi Worker bị chết đột ngột, và hoàn toàn không có khả năng truy vết lịch sử thay đổi của tập dữ liệu.

4.3.4.2. Cách tiếp cận đã thử

- Ghi trực tiếp dữ liệu thành các tệp định dạng Parquet hoặc CSV thuần túy. Tốc độ ghi ban đầu cực kỳ nhanh và nhẹ do không tốn tài nguyên cho lớp quản lý trung gian. Tuy nhiên, hệ thống hoàn toàn mất đi tính bảo đảm ACID. Nếu một Ray Worker bị sập giữa chừng do lỗi tràn bộ nhớ (OOMKilled), tệp Parquet sẽ bị hỏng một nửa, khiến toàn bộ phân hồ dữ liệu đó không thể đọc được nữa mà hệ thống không thể tự động phục hồi (Rollback).

4.3.4.3. Giải pháp cuối cùng

- Quyết định lựa chọn giải pháp Delta Lake làm định dạng lưu trữ tiêu chuẩn cho toàn bộ phân hệ lưu trữ từ tầng Silver trở đi. Logic xử lý của Ray Worker được tích hợp thư viện deltalake để tương tác trực tiếp với MinIO Object Storage.

4.3.4.4. Điểm rút ra

- Hiểu biết kỹ thuật: Xây dựng Data Lakehouse trên hạ tầng phân tán mà thiếu một lớp định dạng bảng (Table Format) như Delta Lake thì bản chất hệ thống đó mới chỉ dừng lại ở một "Vũng lầy dữ liệu" (Data Swamp) vô tổ chức và kém an toàn.

4.3.5 Tích hợp hệ thống

4.3.5.1. Mô tả vấn đề

- Bối cảnh và nền tảng: Các Ray Workers cần nạp trọng số mô hình trước khi tự chơi (Self-play) hoặc huấn luyện (Train).
- Thách thức gặp phải: Nếu chỉ lưu tệp .pt dưới MinIO, hệ thống phân tán không biết đâu là phiên bản mô hình mới nhất, dễ xảy ra tình trạng "Worker A nạp trọng số cũ, Worker B nạp trọng số mới".
- Tác động đến hệ thống: Mạng nơ-ron không hội tụ, hệ thống AI "học mù".

4.3.5.2. Cách tiếp cận đã thử

- Cách 1: Đặt tên file ghi đè liên tục latest_model.pt. Đánh đổi: Không thể quay lui (Rollback) nếu mô hình mới bị suy thoái (Model Degradation).

4.3.5.3. Giải pháp cuối cùng

- Triển khai MLflow Tracking Server kết nối PostgreSQL. MinIO chỉ giữ file vật lý. Khi Ray Worker khởi chạy, nó phải gọi API sang MLflow để truy vấn đường dẫn S3 của mô hình mang thẻ @champion. MLflow đóng vai trò là "Service Discovery" cho tập mô hình.

4.3.5.4. Điểm rút ra

- Luôn phân tách siêu dữ liệu (Metadata) và Dữ liệu vật lý (Artifacts) để đảm bảo tính nhất quán (Consistency) trong phân tán.

4.3.6 Tối ưu hóa hiệu năng

4.3.6.1. Mô tả vấn đề

- Bối cảnh và nền tảng: Tại luồng xử lý `job_2_silver_transform.py`, hệ thống cần phân tích cú pháp (parse) hàng ngàn tệp tin văn bản PGN khổng lồ đang nằm dưới kho lưu trữ MinIO (tầng Bronze). Đặc thù của tác vụ này là

- tải trọng kép: vừa I/O-bound (tốn thời gian tải file qua mạng) vừa CPU-bound (tốn năng lực tính toán để xử lý chuỗi văn bản phức tạp).
- Thách thức gặp phải: Trong hệ thống phân tán, nếu không phân bổ luồng công việc đúng cách, nút chính (Ray Head) rất dễ biến thành một nút thắt cổ chai (Bottleneck). Việc sử dụng hàm chờ kết quả `ray.get()` sai vị trí sẽ vô tình ép một kiến trúc xử lý song song không lồ phải hoạt động một cách tuần tự (Synchronous).
 - Tác động đến hệ thống: Lãng phí hoàn toàn tài nguyên của các máy chủ Worker, kéo dài thời gian chạy tác vụ ETL từ vài phút lên thành nhiều giờ, thậm chí gây sập node trung tâm do quá tải I/O mạng hoặc tràn bộ nhớ.

4.3.6.2. Cách tiếp cận đã thử

- Cách 1: Tải dữ liệu về Head Node rồi mới chia cho Worker. Ray Head đọc toàn bộ nội dung file PGN từ MinIO, cắt thành các chuỗi văn bản lớn rồi mới truyền (pass-by-value) vào các hàm `@ray.remote`. Điều này dẫn đến băng thông mạng nội bộ bị ngốn sạch để đẩy dữ liệu từ Head sang Worker, và Head Node lập tức bị OOMKilled (tràn bộ nhớ) do phải gánh toàn bộ tải I/O.

4.3.6.3. Giải pháp cuối cùng

- Giải pháp tối ưu nhất là triển khai nguyên mẫu Scatter-Gather (Phân tán - Thu thập), tận dụng thiết kế Worker tự trị. Kịch bản điều phối (Orchestrator) chỉ truyền siêu dữ liệu (đường dẫn file) thay vì truyền dữ liệu vật lý. Hiệu suất hệ thống tăng theo cấp số nhân.

4.3.6.4. Điểm rút ra

- Truyền dữ liệu khối lượng lớn qua mạng lưới Ray (giữa Head và Worker) làm giảm hiệu năng. Quy tắc vàng trong Big Data phân tán là: "Hãy đưa năng lực tính toán đến nơi chứa dữ liệu, đừng mang dữ liệu về nơi tính toán" (Truyền đường dẫn, không truyền file).

4.3.7 Giám sát, gỡ lỗi

4.3.7.1. Mô tả vấn đề

- Bối cảnh và nền tảng: Việc chạy các đoạn code Python thông thường chỉ in log (print) ra màn hình console.
- Thách thức gặp phải: Khi code chạy trên cụm Ray phân tán gồm nhiều Pod, kỹ sư không thể xem console của từng máy. Hệ thống trở thành một hộp đen.
- Tác động đến hệ thống: Không thể biết tác vụ nào đang gây nghẽn cổ chai (Bottleneck) hay Worker nào vừa bị chết.

4.3.7.2. Cách tiếp cận đã thử

- Cách 1: Kéo log thủ công bằng lệnh `kubectl logs`. Đánh đổi: Không có tính thời gian thực, khó phân tích.
- Cách 2: Tích hợp bộ Dashboard.

4.3.7.3. Giải pháp cuối cùng

- Định tuyến (Ingress) mở cổng 8265 cho Ray Dashboard để theo dõi Task Topology, mức độ ăn RAM/CPU của từng Node. Mở cổng 5000 cho

MLflow UI để vẽ biểu đồ Loss/Accuracy trực quan. Xây dựng thêm một Dashboard tùy chỉnh ở src/dashboard/ kết nối API cả hai dịch vụ trên.

4.3.7.4. Điểm rút ra

- Khả năng quan sát là yếu tố sống còn.

4.3.8 Mở rộng (Scaling)

4.3.8.1. Mô tả vấn đề

- Bối cảnh và nền tảng: Vòng đời phát triển của một hệ thống AI thường trải qua nhiều môi trường: từ chiếc laptop cá nhân (1 CPU, RAM hạn hẹp) để debug, cho đến cụm Bare-metal Kubernetes sản xuất để huấn luyện thực tế.
- Thách thức gặp phải: Các tham số siêu tham số học máy như số lượng Worker và kích thước lô dữ liệu bị ràng buộc chặt chẽ với giới hạn phần cứng. Nếu khai báo cứng các thông số này, hệ thống sẽ mất đi tính di động.
- Tác động đến hệ thống: Nếu cấu hình cứng cho Server, khi kéo code về chạy trên laptop, máy tính sẽ treo cứng hoặc bị hệ điều hành "giết" ngay lập tức (OOMKilled). Ngược lại, nếu cấu hình cho laptop, khi đẩy lên Server, cụm GPU đắt tiền sẽ bị bỏ không, lãng phí hàng ngàn đô la tài nguyên.

4.3.8.2. Cách tiếp cận đã thử

- Cách 1: Hardcode và sửa tay liên tục. Lập trình viên phải tự nhớ việc comment/uncomment các dòng cấu hình mỗi khi chuyển môi trường. Đánh đổi: Cực kỳ dễ xảy ra lỗi con người, vô tình đẩy cấu hình test lên production làm sập cụm.
- Cách 2: Phụ thuộc hoàn toàn vào file cấu hình (Env/YAML) cho từng môi trường. Đánh đổi: Số lượng file cấu hình phình to.

4.3.8.3. Giải pháp cuối cùng

- Triển khai cơ chế Tự nhận thức phần cứng (Auto-discovery) ngay bên trong mã nguồn khởi tạo (Entrypoint) của Python bằng API của Ray

4.3.8.4. Điểm rút ra

- Khả năng mở rộng của phần mềm không chỉ là việc hệ thống không bị sập khi có nhiều dữ liệu hơn, mà còn là thích ứng được với môi trường khác nhau.

4.3.9 Chất lượng – kiểm thử

4.3.9.1. Mô tả vấn đề

- Bối cảnh: Cụm Ray liên tục xử lý file PGN và đổ vào tầng Silver.
- Thách thức: Hạ tầng phân tán dễ bị lỗi (OOM, rớt mạng). Khi Ray tự động chạy lại (Retry) tác vụ bị lỗi, dữ liệu rất dễ bị ghi trùng lặp.
- Tác động: Làm phình to dung lượng MinIO và khiến mô hình AI học lệch (Overfitting) do học đi học lại một ván cờ.

4.3.9.2. Cách tiếp cận đã thử

- Cách 1: Ghi đè (Overwrite) toàn bộ. Đánh đổi: Mất khả năng tích lũy lịch sử dữ liệu lớn.

- Cách 2: Dùng lệnh MERGE (Upsert). Đánh đổi: Quét toàn bộ bảng hàng triệu dòng gây nghẽn cổ chai I/O nghiêm trọng.

4.3.9.3. Giải pháp cuối cùng

- Sử dụng Polars để khử trùng lặp trực tiếp trên RAM (In-memory) trước khi ghi vật lý:
 - + Tầng 1 (Lọc cục bộ): `df.unique(["game_id"])` xóa ván trùng ngay trong lô dữ liệu hiện tại.
 - + Tầng 2 (Lọc lịch sử): `df.join(existing_ids, how="anti")` loại bỏ những ván đã từng được ghi xuống tầng Silver.

4.3.9.4. Điểm rút ra

- Trong hệ thống phân tán, phải luôn giả định mã nguồn sẽ bị ép "chạy đi chạy lại" do sự cố. Lũy đẳng bằng phép trừ dữ liệu (Anti-join) trên RAM là lớp bảo vệ rẻ và an toàn nhất cho chất lượng Data Lakehouse.

4.3.10 Bảo mật, quản trị

4.3.10.1. Mô tả vấn đề

- Bối cảnh và nền tảng: Quá trình huấn luyện sinh ra các tệp trọng số (Model Weights - file .pt hoặc .npz) có dung lượng từ vài GB đến hàng chục GB. Trong kiến trúc hệ thống MLOps tiêu chuẩn, các tệp này cần được quản lý phiên bản cùng với lịch sử thông số huấn luyện (Loss, Accuracy).
- Thách thức gặp phải: Việc lưu trực tiếp các tệp trọng số không lồ vào bên trong cơ sở dữ liệu của MLflow (PostgreSQL) hoặc thư mục nội bộ của Tracking Server sẽ gây sập hệ thống (OOM hoặc đầy ổ cứng). Hơn nữa, về mặt bảo mật, kỹ sư dữ liệu (Data Engineer) có thể được phép xem lịch sử huấn luyện (biểu đồ Loss) nhưng không được quyền tải về tệp trọng số chứa "chất xám" AI cốt lõi để tránh rủi ro đánh cắp tài sản trí tuệ.
- Tác động đến hệ thống: Nếu không tách biệt được việc lưu trữ, hệ thống vừa phình to công kênh, vừa không thể áp dụng các chính sách bảo mật RBAC (Role-Based Access Control) cho từng nhóm nhân sự chuyên biệt.

4.3.10.2. Cách tiếp cận đã thử

- Cách 1: Ép MLflow log (lưu) trực tiếp toàn bộ thư mục Checkpoint chứa file trọng số vật lý. Đánh đổi: MLflow Tracking Server (vốn được thiết kế Lightweight) bị quá tải mạng. Ổ cứng nội bộ của Pod MLflow bị lấp đầy chỉ sau vài giờ chạy. Ai có quyền xem biểu đồ trên MLflow cũng có quyền tải toàn bộ Model về máy cá nhân.
- Cách 2: Ghi thẳng vào MinIO nhưng không liên kết với MLflow. Đánh đổi: Mất hoàn toàn khả năng quản lý vòng đời (Model Registry). Không biết file `checkpoint_epoch_10.pt` dưới đĩa cứng tương ứng với biểu đồ chất lượng nào trên giao diện quản trị.

4.3.10.3. Giải pháp cuối cùng

- Triển khai kỹ thuật "Con trỏ Mô hình" (Model Pointer). Trọng lượng tính toán nặng được giữ nguyên tại MinIO, trong khi MLflow chỉ lưu giữ siêu dữ liệu mỏng (Thin Metadata) kết nối hai hệ thống.

4.3.10.4. Điểm rút ra

- Không bao giờ lưu trữ tài sản trí tuệ (Model Weights) vào chung một nơi với nhật ký hoạt động (Logs/Metrics). Giải pháp "Con trỏ JSON" tạo tiền đề hoàn hảo để xây dựng kiến trúc Bảo mật bằng sự phân chia trách nhiệm (Security by Compartmentalization).

4.3.11 Chịu lỗi

4.3.11.1. Mô tả vấn đề

- Bối cảnh và nền tảng: Quá trình huấn luyện AlphaZero kéo dài nhiều ngày trên cụm Bare-metal. Việc lưu lại trạng thái mạng nơ-ron (Checkpointing) là bắt buộc để hệ thống có thể đứng dậy tiếp tục học sau khi một Worker bị hệ điều hành "giết" do tràn bộ nhớ (OOMKilled) hoặc mất kết nối mạng.
- Thách thức gặp phải: Tập trọng số (Model Weights) có dung lượng rất lớn. Nếu lưu lại Checkpoint sau mỗi chu kỳ (Epoch), ổ cứng vật lý của MinIO sẽ nhanh chóng bị lấp đầy, tạo ra thảm họa thứ hai: Sập toàn bộ hệ thống lưu trữ (Storage Exhaustion). Mặt khác, nếu chỉ lưu đè mà không lưu một tệp cuối cùng, lỡ hệ thống sập đúng vào lúc mô hình đang bị phân kỳ (Loss tăng cao, AI đánh ngu đi), thì khi phục hồi, ta sẽ phải chạy tiếp từ một phiên bản tồi tệ.
- Tác động đến hệ thống: Mất trắng tệp mô hình tốt nhất đã từng đạt được trong quá khứ, hoặc tốn kém hàng TB đĩa cứng để lưu trữ các phiên bản rác không bao giờ dùng tới.

4.3.11.2. Cách tiếp cận đã thử

- Lưu Checkpoint theo thời gian. An toàn nhưng lãng phí I/O và làm MinIO cạn kiệt dung lượng (Disk Full) chỉ sau một thời gian ngắn.

4.3.11.3. Giải pháp cuối cùng

- Triển khai chiến lược Bảo lưu dựa trên chất lượng (Score-based Selection) thông qua thư viện CheckpointConfig của nền tảng Ray Train

4.3.11.4. Điểm rút ra

- Khả năng chịu lỗi (Fault Tolerance) đích thực không chỉ là việc hệ thống biết cách "chạy lại từ đầu", mà là khả năng tự động phục hồi từ trạng thái tốt nhất.